



Universidad Nacional de Ingeniería
Facultad de Ciencias
Programación Concurrente y Distribuida

Sistema Distribuido de Evaluación de Prestamos

RabbitMQ, Orquestador de Negocio y Bases de Datos
Distribuidas

Estudiante: Cesar Omar Lopez Arteaga
Estudiante: Bryan Pumayauli Evangelista
Estudiante: Ivan Marcelo Urbano Nieves
Proyecto: Despliegue distribuido multi-PC
Middleware: RabbitMQ
Bases de datos: MySQL, PostgreSQL y MariaDB
Lenguajes: Python, Node.js, Java, C#, HTML/JavaScript

Resumen

En el presente proyecto se implemento un sistema distribuido para la evaluacion de prestamos usando RabbitMQ como middleware de mensajeria y una API de negocio centralizada en la PC1. La solucion corrige la arquitectura inicial en la cual los clientes tenian que conocer varias direcciones IP. En esta version, los clientes graficos se conectan unicamente a la API de negocio de la PC1; luego, dicha API orquesta el uso de RabbitMQ y decide que base de datos debe participar en cada operacion.

El sistema fue dividido en varias maquinas: PC1 ejecuta RabbitMQ y el orquestador de negocio; PC2 ejecuta MySQL y un worker Python; PC3 ejecuta PostgreSQL y un worker Node.js; PC4 ejecuta MariaDB y un worker Java; y las PCs cliente ejecutan interfaces graficas en diferentes lenguajes. Las bases de datos se inicializan con datos semilla para permitir pruebas inmediatas de solicitudes aceptadas, rechazadas, observadas y refinanciadas. La comunicacion interna se realiza mediante colas persistentes y mensajes JSON, manteniendo desacoplados los nodos de datos y la logica de decision.

Palabras clave: sistemas distribuidos, RabbitMQ, middleware, prestamos, MySQL, PostgreSQL, MariaDB, Docker, concurrencia, colas.

Índice

1	Introduccion	3
1.1	Problema identificado	3
1.2	Objetivo general	3
1.3	Objetivos especificos	3
2	Arquitectura general del sistema	3
2.1	Resumen de nodos	4
2.2	Despliegue fisico	4
3	Rol de PC1: API de negocio y RabbitMQ	5
3.1	API de negocio	5
3.2	RabbitMQ como middleware	5
4	Protocolo de comunicacion	6
4.1	Solicitud desde el cliente	7
4.2	Respuesta final al cliente	8
5	Modelo de datos distribuido	8
5.1	MySQL en PC2	8
5.2	PostgreSQL en PC3	8
5.3	MariaDB en PC4	8
5.4	Datos semilla	9
6	Logica de negocio	9
6.1	Variables consideradas	10
6.2	Reglas de decision	11
7	Implementacion por nodo	11
7.1	PC1: RabbitMQ y API de negocio	11
7.2	PC2: MySQL y Worker Python	11
7.3	PC3: PostgreSQL y Worker Node.js	12
7.4	PC4: MariaDB y Worker Java	12
8	Clientes graficos	12
9	Validacion del sistema	12
9.1	Validacion de PC1	12

9.2	Validacion desde PC2, PC3 y PC4	13
9.3	Prueba funcional	13
9.4	Consultas SQL de verificacion	13
10	Prueba de concurrencia	13
11	Manejo de errores y tolerancia	13
12	Conclusiones	14
13	Trabajo futuro	14
Anexo A:	Comandos de despliegue rapido	15

1 Introduccion

El objetivo del proyecto es construir un sistema distribuido que permita registrar solicitudes de prestamo, procesarlas mediante una logica de negocio y almacenar los resultados en bases de datos separadas. La solucion se diseno considerando un escenario realista donde diferentes servicios se encuentran desplegados en maquinas independientes dentro de una misma red local.

A diferencia de una aplicacion monolitica, este sistema separa la captura de solicitudes, el almacenamiento, la evaluacion del riesgo y la persistencia de decisiones. Para lograr esta separacion se utiliza RabbitMQ como middleware central. Sin embargo, RabbitMQ no contiene la logica de negocio; su funcion es transportar mensajes entre los servicios. La decision de que base de datos usar y como calcular el riesgo se ubica en la API de negocio de PC1.

1.1 Problema identificado

En una arquitectura distribuida mal planteada, los clientes podrian necesitar conectarse a multiples servidores: uno para MySQL, otro para PostgreSQL, otro para MariaDB y otro para RabbitMQ. Esto genera acoplamiento, dificultad de mantenimiento y mayor probabilidad de error al cambiar direcciones IP.

Por este motivo, se replanteo la solucion para que el cliente se conecte solamente a PC1. La PC1 actua como punto de entrada del sistema, recibe las solicitudes y coordina la comunicacion con los demas nodos mediante RabbitMQ.

1.2 Objetivo general

Implementar un sistema distribuido de evaluacion de prestamos con una API de negocio centralizada en PC1, RabbitMQ como middleware de comunicacion, bases de datos distribuidas y clientes graficos capaces de conectarse desde diferentes PCs.

1.3 Objetivos especificos

- Desplegar RabbitMQ y la API de negocio en PC1.
- Desplegar MySQL y un worker Python en PC2.
- Desplegar PostgreSQL y un worker Node.js en PC3.
- Desplegar MariaDB y un worker Java en PC4.
- Implementar clientes graficos en diferentes lenguajes que se conecten solo a PC1.
- Inicializar las bases de datos con informacion de prueba para validar la logica de negocio.
- Verificar el funcionamiento mediante endpoints HTTP, colas RabbitMQ y consultas SQL.

2 Arquitectura general del sistema

La arquitectura final utiliza a PC1 como punto unico de entrada para los clientes. El cliente grafico no conoce las IPs internas de PC2, PC3 ni PC4. Solo conoce la IP de PC1 y envia solicitudes HTTP a la API de negocio. Luego, PC1 decide internamente que colas publicar y que bases de datos deben intervenir.

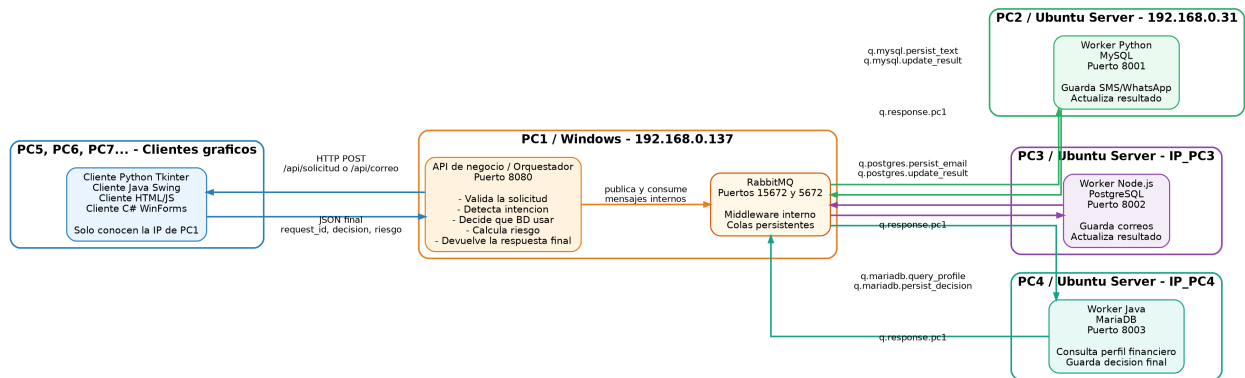


Figura 1: Arquitectura final del sistema distribuido con PC1 como orquestador de negocio.

2.1 Resumen de nodos

Cuadro 1: Distribucion fisica del sistema por computadora

PC	IP	Servicio	Funcion principal
PC1 / Windows	192.168.0.137	RabbitMQ + API de negocio	Recibe las solicitudes de los clientes, orquesta la logica y usa RabbitMQ para comunicarse con los workers.
PC2 / Ubuntu Server	192.168.0.31	MySQL + Worker Python	Guarda mensajes SMS/WhatsApp y actualiza el resultado de solicitudes de texto.
PC3 / Ubuntu Server	IP_PC3	PostgreSQL + Worker Node.js	Guarda correos electronicos y actualiza el resultado de solicitudes provenientes de correo.
PC4 / Ubuntu Server	IP_PC4	MariaDB + Worker Java	Contiene usuarios, cuentas, prestamos y decisiones finales.
PC5, PC6, PC7	varias	Clientes graficos	Envio de solicitudes mediante interfaces graficas.

2.2 Despliegue fisico

La red local utilizada corresponde al segmento 192.168.0.0/24. En el caso de las maquinas virtuales Ubuntu Server se recomienda configurar VirtualBox en modo *adaptador puente*, para que cada VM reciba una IP propia visible desde Windows y desde los demas nodos.

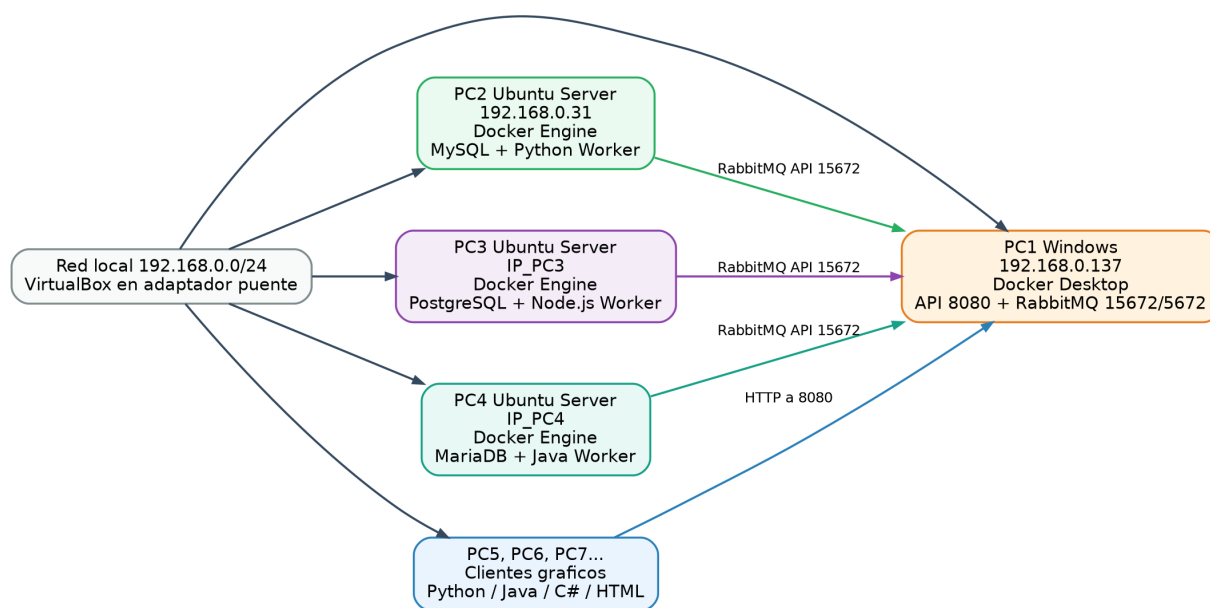


Figura 2: Despliegue físico del sistema sobre varias PCs o máquinas virtuales.

3 Rol de PC1: API de negocio y RabbitMQ

PC1 tiene dos componentes diferentes. El primero es RabbitMQ, que se encarga de recibir, almacenar temporalmente y entregar mensajes entre los servicios. El segundo es la API de negocio, que representa el verdadero punto de decisión del sistema.

3.1 API de negocio

La API de negocio se implementa en Python usando un servidor HTTP concurrente basado en `ThreadingHTTPServer`. Esta API recibe las solicitudes de los clientes por HTTP, genera un identificador `request_id`, decide si la solicitud corresponde a mensaje o correo, solicita información financiera a MariaDB y calcula el riesgo crediticio.

Cuadro 2: Endpoints principales de PC1

Endpoint	Metodo	Funcion
/api/health	GET	Verifica que la API de negocio este activa.
/api/solicitud	POST	Recibe solicitudes SMS/WhatsApp desde los clientes graficos.
/api/correo	POST	Recibe solicitudes formuladas como correo electronico.
/api/requests	GET	Lista las ultimas solicitudes procesadas.
/api/requests/{id}	GET	Consulta el estado de una solicitud mediante su <code>request_id</code> .
/api/metrics	GET	Muestra metricas de solicitudes recibidas, procesadas y errores.

3.2 RabbitMQ como middleware

RabbitMQ no evalúa préstamos, no calcula riesgo y no decide qué base de datos usar. Su función es actuar como intermediario confiable entre PC1 y los workers. La API de negocio publica mensajes en colas, y los workers de PC2, PC3 y PC4 los consumen según su responsabilidad.

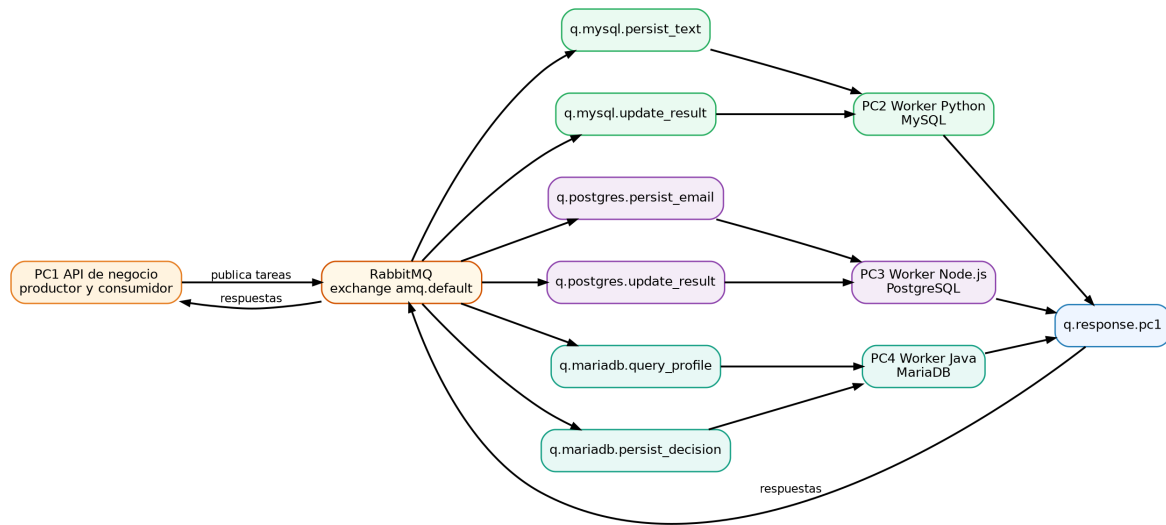


Figura 3: Mapa de colas usadas por RabbitMQ en la arquitectura final.

Cuadro 3: Colas RabbitMQ utilizadas por el sistema

Cola	Consumidor	Uso
q.mysql.persist_text	PC2 / Python	Guardar solicitud SMS/WhatsApp en MySQL.
q.mysql.update_result	PC2 / Python	Actualizar estado, decision y riesgo en MySQL.
q.postgres.persist_email	PC3 / Node.js	Guardar correo electronico en PostgreSQL.
q.postgres.update_result	PC3 / Node.js	Actualizar resultado del correo en PostgreSQL.
q.mariadb.query_profile	PC4 / Java	Consultar perfil financiero del usuario.
q.mariadb.persist_decision	PC4 / Java	Guardar decision final en MariaDB.
q.response.pc1	PC1 / API	Devolver confirmaciones y resultados hacia el orquestador.

4 Protocolo de comunicacion

El protocolo de comunicacion se basa en mensajes JSON intercambiados por HTTP entre cliente y PC1, y por RabbitMQ entre PC1 y los workers. El cliente nunca interactua directamente con RabbitMQ, porque eso expondria detalles internos del sistema.

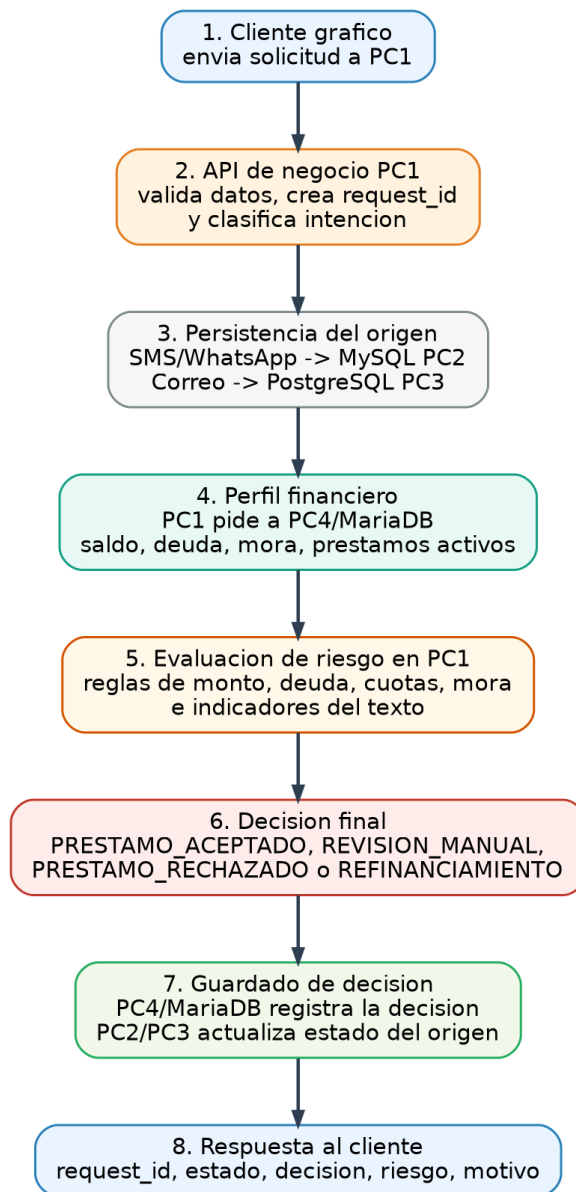


Figura 4: Secuencia de procesamiento de una solicitud de prestamo.

4.1 Solicitud desde el cliente

Para una solicitud de prestamo enviada desde la interfaz grafica, el cliente realiza una peticion HTTP a PC1:

```
1 POST http://192.168.0.137:8080/api/solicitud
2 {
3   "id_usuario": 1,
4   "canal": "WhatsApp",
5   "tipo": "solicitud",
6   "contenido": "Solicito un prestamo de 5500 soles, tengo sueldo estable"
7 }
```

Listing 1: Ejemplo de solicitud enviada por el cliente a PC1.

4.2 Respuesta final al cliente

La API de negocio espera las confirmaciones de los workers y devuelve al cliente una respuesta consolidada:

```

1 {
2   "ok": true,
3   "result": {
4     "request_id": "REQ-20260604152210-a81f2c33",
5     "estado": "PROCESADO",
6     "decision": "PRESTAMO_ACEPTADO",
7     "riesgo": 23,
8     "motivo": "monto compatible con saldo; texto indica ingresos estables"
9   }
10 }
```

Listing 2: Ejemplo de respuesta final generada por PC1.

5 Modelo de datos distribuido

El sistema usa tres motores de base de datos diferentes. Esta separacion permite simular un entorno distribuido donde cada nodo conserva una responsabilidad especifica.

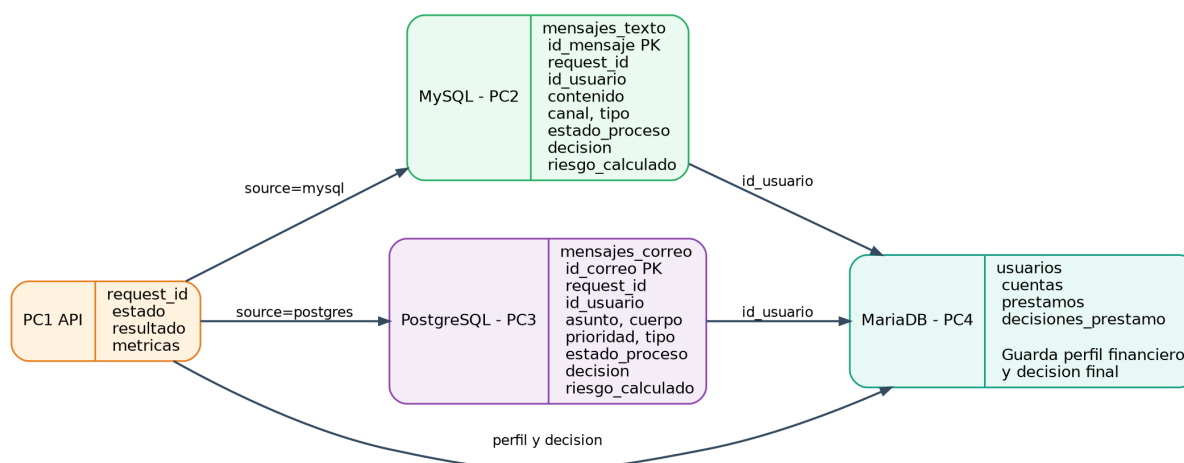


Figura 5: Modelo de datos distribuido por motor de base de datos.

5.1 MySQL en PC2

MySQL almacena las solicitudes de texto recibidas por canales como SMS o WhatsApp. La tabla principal es `mensajes_texto`. Cada registro queda asociado a un `request_id`, al usuario y al resultado calculado por la API de negocio.

5.2 PostgreSQL en PC3

PostgreSQL almacena correos electronicos mediante la tabla `mensajes_correo`. En este caso, los campos principales son `asunto`, `cuerpo`, `prioridad`, `tipo`, `estado_proceso`, `decision` y `riesgo_calculado`.

5.3 MariaDB en PC4

MariaDB contiene la informacion financiera base: `usuarios`, `cuentas`, `prestamos` y `decisiones_prestamo`. Esta base es la fuente principal para calcular el perfil de riesgo del cliente.

5.4 Datos semilla

Las bases de datos se inicializan con datos de prueba para que la logica pueda ejecutarse sin registrar manualmente toda la informacion financiera. La semilla incluye usuarios con diferentes condiciones: sueldo estable, cuentas con saldo alto, prestamos activos, mora y cuotas vencidas. Esto permite validar distintos resultados, como aprobacion, revision manual, rechazo y refinanciamiento.

Cuadro 4: Ejemplos de escenarios cargados como datos iniciales

Escenario	Datos relevantes	Resultado esperado
Cliente solvente	Saldo suficiente, texto con sueldo estable, sin mora	Prestamo aceptado o riesgo bajo.
Cliente con deuda	Prestamos activos y monto elevado	Revision manual o riesgo medio.
Cliente moroso	Cuotas vencidas, dias de mora y texto con urgencia	Prestamo rechazado o riesgo alto.
Refinanciamiento	Solicitud explicita de refinanciar deuda	Propuesta u observacion de refinanciamiento.
Consulta	Texto orientado a consultar estado del prestamo	Consulta registrada.

6 Logica de negocio

La logica de negocio se ubica en PC1. Este criterio fue adoptado para que el cliente no decida que base usar y para que RabbitMQ no sea confundido con un modulo de decision. La API de negocio orquesta las operaciones y aplica las reglas de evaluacion.

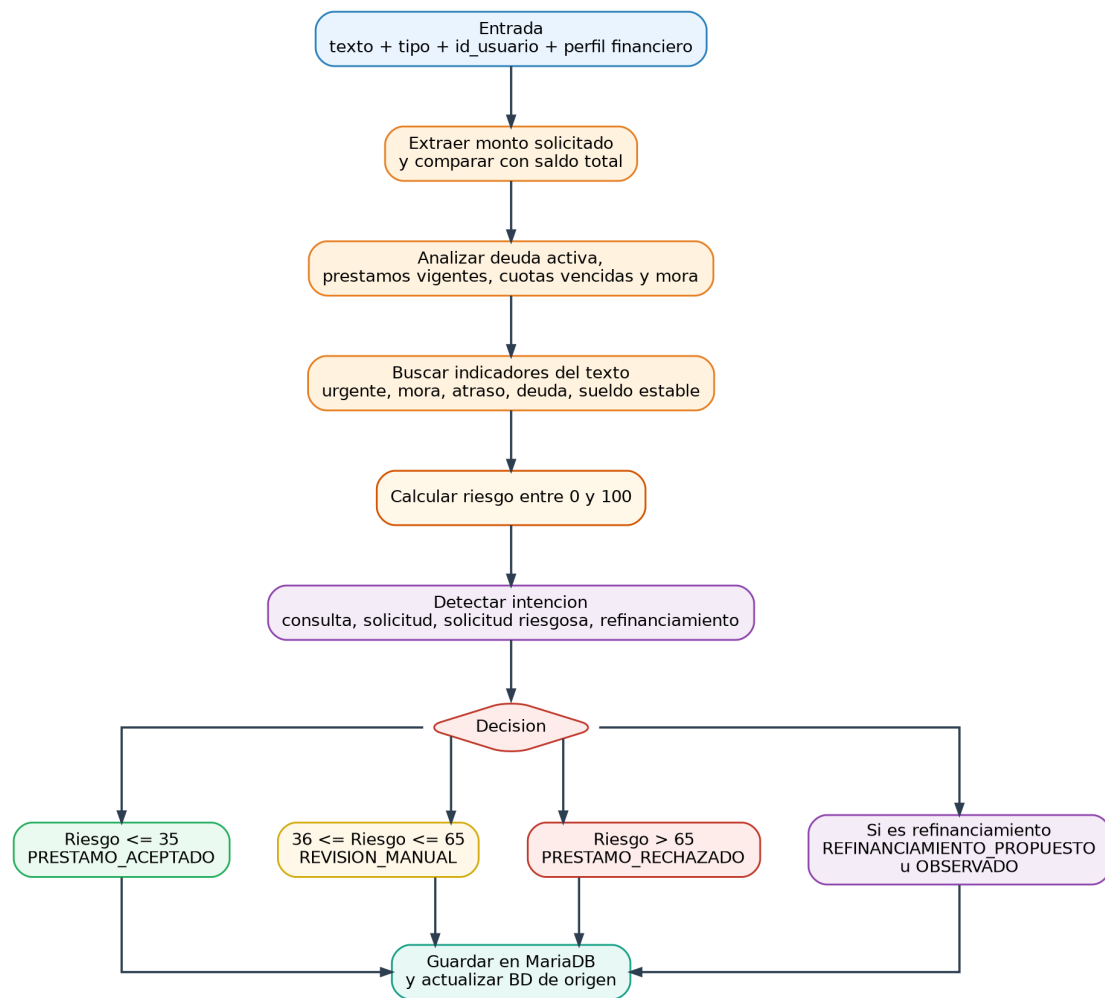


Figura 6: Flujo de calculo de riesgo y decision de negocio.

6.1 Variables consideradas

La evaluacion considera variables textuales y financieras:

- Monto solicitado.
- Saldo total del usuario.
- Prestamos activos.
- Deuda total vigente.
- Numero de cuotas vencidas.
- Dias maximos de mora.
- Indicadores del texto: *urgente, deuda, mora, atraso, sueldo estable*.
- Tipo de solicitud: solicitud, consulta o refinanciamiento.

6.2 Reglas de decision

El riesgo se normaliza en el rango de 0 a 100. Mientras mayor sea el valor, mayor es la posibilidad de rechazo u observacion.

Cuadro 5: Reglas de decision usadas por la API de negocio

Condicion	Decision	Interpretacion
Riesgo menor o igual a 35	PRESTAMO_ACEPTADO	El perfil financiero es favorable.
Riesgo entre 36 y 65	REVISION_MANUAL	El caso requiere revision adicional.
Riesgo mayor a 65	PRESTAMO_RECHAZADO	El perfil presenta alto riesgo.
Texto de refinanciamiento	REFINANCIAMIENTO_PROPUESTO o REFINANCIAMIENTO_OBSERVADO	El sistema deriva el caso a refinanciamiento.
Texto de consulta	CONSULTA_REGISTRADA	La solicitud no crea prestamo nuevo, solo registra consulta.

7 Implementacion por nodo

7.1 PC1: RabbitMQ y API de negocio

En PC1 se ejecuta Docker Desktop. El archivo de despliegue se encuentra en `deploy/pc1_rabbitmq_business`. Los puertos principales son 8080 para la API de negocio, 15672 para el panel web de RabbitMQ y 5672 para AMQP.

```
1 cd D:\proyecto_prestamos_distribuido_v8_pc1_orquestador\deploy\pc1_rabbitmq_business
2 copy .env.example .env
3 docker compose -f docker-compose.pc1.yml up -d --build
```

Listing 3: Despliegue de PC1 en Windows.

Para permitir que las PCs Ubuntu accedan a RabbitMQ y a la API, se habilitan reglas de firewall:

```
1 8080 # API de negocio
2 15672 # RabbitMQ Management
3 5672 # RabbitMQ AMQP
```

Listing 4: Puertos principales a habilitar en Windows.

7.2 PC2: MySQL y Worker Python

PC2 almacena solicitudes de texto. El worker Python consume tareas desde RabbitMQ, persiste mensajes en MySQL y devuelve confirmaciones a PC1.

```
1 cd ~/proyecto_prestamos_distribuido_v8_pc1_orquestador/deploy/pc2_mysql_node
2 cp .env.example .env
3 nano .env
4 # RABBIT_HOST=192.168.0.137
5 docker compose -f docker-compose.pc2.yml up -d --build
```

Listing 5: Despliegue de PC2 en Ubuntu Server.

7.3 PC3: PostgreSQL y Worker Node.js

PC3 almacena solicitudes de correo. Este nodo usa Node.js para consumir mensajes de RabbitMQ y ejecutar comandos sobre PostgreSQL.

```

1 cd ~/proyecto_prestamos_distribuido_v8_pc1_orquestador/deploy/pc3_postgres_node
2 cp .env.example .env
3 nano .env
4 # RABBIT_HOST=192.168.0.137
5 docker compose -f docker-compose.pc3.yml up -d --build

```

Listing 6: Despliegue de PC3 en Ubuntu Server.

7.4 PC4: MariaDB y Worker Java

PC4 almacena datos financieros y decisiones. El worker Java atiende dos tipos de tareas: consultar perfil financiero y persistir decisiones finales.

```

1 cd ~/proyecto_prestamos_distribuido_v8_pc1_orquestador/deploy/pc4_mariadb_worker
2 cp .env.example .env
3 nano .env
4 # RABBIT_HOST=192.168.0.137
5 docker compose -f docker-compose.pc4.yml up -d --build

```

Listing 7: Despliegue de PC4 en Ubuntu Server.

8 Clientes graficos

Los clientes graficos se ejecutan desde PC5, PC6, PC7 u otras maquinas. En esta arquitectura todos los clientes se conectan solamente a PC1.

Cuadro 6: Clientes incluidos en el proyecto

Cliente	Lenguaje	Ejecucion
Python Tkinter	Python	python client_gui_python.py
Java Swing	Java	javac PrestamosSwingClient.java y java PrestamosSwingClient
HTML/JS	JavaScript	Abrir index.html en navegador.
C# WinForms	C#	dotnet run

En el cliente se debe configurar solo la IP de PC1:

```

1 API PC1: http://192.168.0.137:8080

```

Listing 8: Direccion que usan los clientes.

9 Validacion del sistema

9.1 Validacion de PC1

```

1 curl.exe http://localhost:8080/api/health
2 curl.exe http://192.168.0.137:8080/api/health
3 curl.exe http://192.168.0.137:15672

```

Listing 9: Validar API y RabbitMQ desde Windows.

9.2 Validacion desde PC2, PC3 y PC4

```
1 curl -u admin:adminpass http://192.168.0.137:15672/api/overview
```

Listing 10: Verificar conexion de los workers hacia RabbitMQ.

9.3 Prueba funcional

```
1 curl -X POST http://192.168.0.137:8080/api/solicitud \  
2 -H "Content-Type: application/json" \  
3 -d '{"id_usuario":1,"canal":"WhatsApp","tipo":"solicitud","contenido":"Solicito un  
    prestamo de 5500 soles, tengo sueldo estable"}'
```

Listing 11: Prueba de solicitud usando curl.

9.4 Consultas SQL de verificacion

```
1 docker exec -it mysql-prestamos mysql --default-character-set=utf8mb4 \  
2 -uappuser -papppass prestamos_texto \  
3 -e "SELECT id_mensaje,request_id,id_usuario,estado_proceso,decision,riesgo_calculado FROM  
    mensajes_texto ORDER BY id_mensaje DESC LIMIT 10;"
```

Listing 12: Validar MySQL en PC2.

```
1 docker exec -it postgres-prestamos psql -U appuser -d prestamos_correo \  
2 -c "SELECT id_correo,request_id,id_usuario,estado_proceso,decision,riesgo_calculado FROM  
3 mensajes_correo ORDER BY id_correo DESC LIMIT 10;"
```

Listing 13: Validar PostgreSQL en PC3.

```
1 docker exec -it mariadb-prestamos mariadb --default-character-set=utf8mb4 \  
2 -uappuser -papppass prestamos_finanzas \  
3 -e "SELECT id_decision,request_id,source,id_usuario,decision,riesgo FROM  
    decisiones_prestamo ORDER BY id_decision DESC LIMIT 10;"
```

Listing 14: Validar MariaDB en PC4.

10 Prueba de concurrencia

La prueba de rendimiento se realiza enviando 1000 solicitudes desde el cliente de prueba. El script usa multiples hilos para simular concurrencia y medir el tiempo total de procesamiento.

```
1 python tools/perf_1000.py --host 192.168.0.137 --n 1000 --workers 20
```

Listing 15: Prueba concurrente de 1000 solicitudes.

Esta prueba permite observar si la API de negocio, RabbitMQ y los workers mantienen consistencia de datos. Al finalizar se pueden revisar las tablas de MySQL, PostgreSQL y MariaDB para confirmar que no existen registros duplicados ni solicitudes sin decision.

11 Manejo de errores y tolerancia

La version mejorada contempla validaciones y diagnostico por componentes. Si un worker no puede conectarse a RabbitMQ, el error se identifica revisando sus logs. Si la API de PC1 no responde desde otras PCs, se revisa el firewall de Windows y la configuracion del adaptador puente de VirtualBox.

Cuadro 7: Errores comunes y correccion

Error	Causa probable	Solucion
Connection timed out hacia PC1	Firewall de Windows bloquea el puerto	Abrir puertos 8080, 15672 y 5672.
RabbitMQ no disponible en worker	RABBIT_HOST incorrecto o RabbitMQ apagado	Corregir <code>.env</code> y verificar <code>/api/overview</code> .
Cliente no recibe respuesta	PC4 no esta activo o no guarda decision	Levantar MariaDB worker y revisar logs.
BD no muestra datos semilla	Volumen Docker antiguo reutilizado	Ejecutar <code>docker compose down -v</code> solo si se desea reiniciar la BD.
VM no es visible desde Windows	VirtualBox en modo NAT	Cambiar a adaptador puente.

12 Conclusiones

- Se implemento una arquitectura distribuida en la cual los clientes se conectan unicamente a PC1, reduciendo acoplamiento y simplificando el despliegue.
- RabbitMQ actua como middleware interno, no como modulo de negocio. La logica de decision se ubica en la API de negocio de PC1.
- MySQL, PostgreSQL y MariaDB cumplen responsabilidades distintas y se despliegan en maquinas separadas.
- La existencia de datos semilla permite validar inmediatamente la logica de riesgo y las decisiones de prestamo.
- La division por workers facilita el crecimiento del sistema, ya que cada nodo puede escalar o modificarse sin cambiar los clientes.

13 Trabajo futuro

Como mejora futura se podria reemplazar la evaluacion basada en reglas por un modelo de aprendizaje automatico, agregar autenticacion de clientes, registrar auditoria por solicitud, incorporar HTTPS y usar colas de respuesta con correlacion AMQP nativa. Tambien se podria desplegar RabbitMQ en una maquina Linux dedicada o en la nube para mejorar disponibilidad.

Anexo A: Comandos de despliegue rapido

PC1

```
1 cd deploy/pc1_rabbitmq_business
2 cp .env.example .env
3 docker compose -f docker-compose.pc1.yml up -d --build
```

PC2

```
1 cd deploy/pc2_mysql_node
2 cp .env.example .env
3 # editar RABBIT_HOST=192.168.0.137
4 docker compose -f docker-compose.pc2.yml up -d --build
```

PC3

```
1 cd deploy/pc3_postgres_node
2 cp .env.example .env
3 # editar RABBIT_HOST=192.168.0.137
4 docker compose -f docker-compose.pc3.yml up -d --build
```

PC4

```
1 cd deploy/pc4_mariadb_worker
2 cp .env.example .env
3 # editar RABBIT_HOST=192.168.0.137
4 docker compose -f docker-compose.pc4.yml up -d --build
```